potenzielles memory system, potenzielle Ermöglichung von einem Go Transpiler

Status: Analyse, nicht implementiert. Diese Notiz beschreibt einen möglichen Speicher-Tier und was er für einen Go-Transpiler bedeuten könnte. Sie ändert weder `Tensor`, `Store`, den Scheduler noch die Persistenz. Die bestehenden Invarianten bleiben unangetastet, bis eine eigene Entscheidung sie ausdrücklich ändert.

Befund: der heutige Tensor hat keine Arena

Der Tensor ist ein reiner Wert:

- `Tensor = { Data: double[]; Shape: int[] }` (`types.fs:36`).
- `Store.Lexicon: Map<EntryIdentity, Tensor>` (`types.fs:42`).
- `Store.addTensor` hängt an und vergibt die nächste `EntryIdentity` (`store.fs:17`).
- `Tensor.append` und `Tensor.concat` allokieren per `Array.append` frische managed Arrays (`tensor.fs:76,97,107`).
- `EvaluationState.Store` ist Teil des Ausführungszustands (`types.fs:425`).
- `Actor.State` ist ein vollständiger `EvaluationState` (`types.fs:481`).

Entscheidend ist der Scheduler:

```
{ nextActor.State with
    Store = store
    SchedulerState = sched' }
```

`switchTo` überschreibt den Store des nächsten Actors mit **einem gemeinsamen, live threading Store** (`scheduler.fs:47-49`). `yieldControl` reicht genau diesen Store weiter (`scheduler.fs:253-280`). Kinder erben ihn über `freshChildState parentState` (`scheduler.fs:315-335,383-385`). Es gibt also heute **einen append-only Image-Store pro Prozess**, nicht einen Store pro Actor.

Warum „Arena pro Handler“ damit kollidiert

1. **Gemeinsamer Store statt Actor-Besitz.** Ein per-Actor-Arena-Modell verlangt, dass `switchTo` nicht mehr global `Store = store` setzt, sondern die Arena des Ziel-Actors auswählt und die des laufenden Actors sichert. Das ist eine Scheduler- und State-Änderung, keine lokale Allokator-Änderung.

2. **Mailboxen sind store-resident.** `createMailbox` legt eine Channel-Identity im Store an (`scheduler.fs:339`); `deliverMessage` hängt Nachrichten als Store-Einträge in die Queue (`scheduler.fs:459`); `readChannelState` und `appendChannelState` lesen und schreiben diese Queue im Store (`store.fs:67,83`). Eine Nachricht in der Sender-Arena würde beim Reset des Senders dangling. Nachrichten müssen daher entweder im Image bleiben oder beim Senden in eine langlebigere Region kopiert werden.

3. **`yield`, `halt`, `resume`.** `yieldControl` snapshottet den gesamten `EvaluationState`; `haltCurrent` behält Programm und Stack ausdrücklich für `resume` (`scheduler.fs:282-311`). Ein Reset am „Handler-Ende“ ist damit nicht automatisch ein Reset am Turn-Ende: Ein Actor kann mitten im Handler suspenden. Eine Arena pro **Nachricht** würde Stack- und Continuation-Werte invalidieren. Eine Arena pro **Actor-Inkarnation** ist mit dem heutigen Modell verträglich; eine Arena pro **Turn** nur mit Promotion an jeder Yield- und Halt-Grenze.

4. **Stabile Identitäten und Persistenz.** `EntryIdentity` ist eine Position im append-only Image; `Persistence` serialisiert `Lexicon`, `TextCache` und `NextIndex` (`persistence.fs:86-95`). Arena-Werte sind Ausführungszustand, nicht Image-Daten. Ein Wert, der persistiert, an einen anderen

Actor gesendet oder in einer Definition gehalten wird, muss vorher in den Store **promotet** werden. Sonst gibt es zwei Adressräume mit unterschiedlicher Lebensdauer und keine klare Regel, wann eine `EntryIdentity` gültig ist.

5. **Immutability gegen Bump-Allocator.** `EvaluationState` wird bei `yield`, `halt` und Supervision als unveränderlicher Snapshot gehalten. Ein mutabler Bump-Allocator in diesem Record würde von mehreren Snapshots aliasiert. Entweder liegt die Arena **außerhalb** des gespeicherten `EvaluationState` (Runtime-Ressource pro Inkarnation), oder ein Reset erzeugt eine neue leere Region statt sie zu mutieren. „Reset in O(1)“ und „immutable State-Snapshots“ sind Spannungen, die man explizit auflösen muss.
6. **Managed Arrays sind kein echter Bump-Allocator.** `double[]` bleibt GC-verwaltet. Eine Arena aus managed Arrays reduziert Allokationsdruck, verhindert aber keine GC. Für „ohne GC“ braucht man unmanaged Speicher (`NativeMemory`, `MemoryPool`) und eine Tensor-View mit Pointer, Offset und Länge. Das ändert `Tensor`, alle Tensor-Funktionen, Persistenz und die AOT-Grenzen.
7. **Reclaim und Compaction.** `Reclaim.report` scannt Tensor-Daten numerisch nach möglichen Referenzen (`reclaim.fs:17-26`); `compact` filtert das Lexicon (`reclaim.fs:173-179`). Arena-Tensoren dürfen nicht in dieses Lexicon geraten, sonst verwechselt der Scanner Daten mit Referenzen. Arena-Reset und Image-Compaction wären zwei getrennte Mechanismen.

Zwei Finalisierungsbegriffe, zwei Mechanismen

Für Scale müssen **Speicherrückgewinnung** und **Seiteneffekte** getrennt bleiben.

- **Speicher:** Eine lineare Epoche ist der Token; die Referenzen sind affin. Epochenwerte dürfen fallen gelassen, aber nicht kopiert oder dupliziert werden.
- **Seiteneffekte:** Ein linearer Finalizer-Token schützt `close`, `flush` und `commit`; ein Scope-Bracket führt ihn in jedem Exit-Pfad aus.

Ein rein affines System ohne Epochen- oder Finalizer-Token kann keine garantierte Finalisierung bieten: Ein affiner Wert darf still verworfen werden, und dann läuft kein Destruktor.

Lineare Epoche plus affine Referenzen

```
Epoch      : linear   (genau einmal schließen)
TensorRef  : affin    (darf fallen gelassen werden, aber nicht dupliziert)
ImageRef   : stabil   (EntryIdentity im append-only Store)
```

Konkret für Scale:

- Eine `Epoch` besitzt eine **logische Menge** von `EntryIdentity`s, keine zusammenhängende Range. Mehrere Actors allokieren interleaved in denselben Store; eine echte Bump-Range pro Actor gäbe es nur mit separaten Sub-Stores.
- `Store.NextIndex` bleibt monoton. Beim Epochenende werden die Einträge aus `Lexicon` und `TextCache` entfernt; die verbleibenden `EntryIdentity`s behalten ihre Werte. Es wird nicht renummeriert. Löcher im `Map`-basierten Lexicon sind unproblematisch.
- Das ist mit `Reclaim.compact` vereinbar: `compact` filtert bereits `Lexicon` und `TextCache`, ohne `NextIndex` zu ändern (`reclaim.fs:173-179`). Eine Epoche kann entweder exakt ihre eigenen IDs filtern oder `Reclaim.report` mit ihren Roots plus den globalen Roots laufen lassen.
- **Promotion ist die Grenze.** Sobald ein Wert die Epoche verlassen soll (Mailbox, anderer Actor, persistierter Zustand, Definition oder ein über `yield` hinaus lebender Stack), wird er in den Image-Store kopiert und erhält eine stabile `EntryIdentity`. Danach ist er kein Epochenwert mehr.
- **Escape ist der Feind.** Ein globaler Root, der auf eine Epochen-ID zeigt, ist ein Affinitätsbruch. Entweder wird vorher promotet, oder der Zugriff muss über eine Generation und Epochen-ID als

ungültig erkannt werden. In Scales dynamischem Ownership-Modell (W9, D30) lässt sich das zur Laufzeit prüfen; statisch wäre es stärker, ist aber ein neues Typsystem.

Damit gilt:

> Eine `EntryIdentity` ist entweder **image-stabil** oder **ephemer**. Ephemere IDs > dürfen niemals in `Store.Lexicon`, `TextCache`, Mailbox-Queues, persistierte > Daten oder fremde Actor-Stacks gelangen, ohne vorher promotet zu werden.

Das ist genau die Grenze, die heute fehlt: Der gemeinsame Store und die store-residenten Mailboxen (`store.fs:67,83`, `scheduler.fs:459`) machen jede naive Actor-Arena sofort unsicher.

Scope-Bracket plus linearer Finalizer-Token
Für `close`, `flush` und `commit` reicht eine Speicherepoche nicht. Diese Operationen haben Seiteneffekte und müssen **genau einmal** laufen, auch auf Fehlerpfaden.

```
FinalizerToken : linear
bracket        : token -> body -> result   (release in jedem Exit-Pfad)
```

- `bracket` öffnet die Ressource, führt den Body aus und ruft `release` im Normal-, Fehler- und Abbruchpfad auf.
- Der Token ist linear: Er kann nicht implizit gedroppt werden; `release` ist die einzige Konsumform.
- Die Reihenfolge ist explizit: `flush` vor `commit`, `commit` vor `close`, `close` idempotent.
- Der Token gehört **nicht** in das persistierte Image. Er ist Ausführungszustand, wie Ownership, Scheduler und Actor-Handles.
- Bei Actor-Restart oder Termination muss der Supervisor den Token finalisieren. Ein Token im alten Actor-Snapshot reicht nicht, weil der Snapshot nicht serialisiert wird und ein Restart eine neue Inkarnation startet.
- Für Prozessabbruch gilt die Garantie nicht. Dort braucht es ein dauerhaftes Intent-Log und einen idempotenten Finalizer (At-least-once), nicht nur `bracket`.

In Scale liegen die echten OS-Ressourcen heute an der Host-Grenze (Datei lesen und schreiben, Persistenz), nicht im Kern. Channels sind logische, store-residente Zustände, keine OS-Handles. Deshalb ist der pragmatische Ort für `bracket` die Host- und Entry-Grenze; sobald Actors echte Ressourcen halten, braucht der Supervisor eine finale `FinalizerTable` als Ausführungszustand.

Die minimale, konsistente Scale-Variante

Die verbindliche Terminologie und der Elixir-fokussierte Zielzustand stehen in `docs/memory-model.typ`; der Go-Transpiler ist verzögert und treibt dieses Modell nicht.

1. **Image bleibt Autorität** für alles, was Actor-Grenzen, Mailboxen, Persistenz oder Compaction überlebt.
2. **Epoche pro Actor-Inkarnation** für ephemere Tensoren; Reset bei Termination oder Restart, nicht bei jedem `yield`. Bei einer Epoche pro Turn müsste jeder `yield` und `halt` Werte promoten.
3. **Promotion an jeder Grenze**: `send`, `publish`, `persist`, fremder Stack, Definition.
4. **Lineare Epoche** mit genau einem `close`; Generation im Handle gegen Use-after-close.
5. **Separater Finalizer-Token** für Seiteneffekte; `bracket` an der Host-Grenze, Supervisor-Finalisierung bei Actor-Ende.
6. **`Reclaim.compact` bleibt der globale Aufräumer**; die Epoche ist eine zusätzliche, kürzere Lebensdauer, kein Ersatz dafür.

Potenzielle Ermöglichung eines Go-Transpilers

Ein Go-Transpiler nach Scale wäre nicht nur eine Syntaxübersetzung. Die relevanten Go-Semantiken sind:

- **Heap und GC.** Go-Programme allokieren fast alles auf einem tracing-GC-Heap; Zeiger dürfen frei zwischen Goroutinen geteilt werden. Eine affine Epochenarchitektur allein kann das nicht abbilden.
- **Goroutinen.** `go f()` ist leichtgewichtige Nebenläufigkeit mit eigener Ausführung und einem gemeinsamen Speicher. Scale-Actors sind isolierte Zustände mit Nachrichten; ein direktes `go`-Mapping ist nur für eine Teilmenge möglich.
- **Channels.** Go-Channels sind typisierte, synchronisierende Queues. Scales store-residente Channels sind eine naheliegende Zielstruktur, aber Pufferung, `close`-Semantik und `select` müssen exakt abgebildet werden.
- **`defer`.** Go führt zurückgestellte Aufrufe LIFO beim Funktionsende aus, auch bei Panic. Das entspricht einem Stack von Finalizer-Tokens plus `bracket`.
- **`panic` und `recover`.** Sie sind Kontrollfluss über normale Grenzen hinweg; in Scale müssten sie auf Conditions, Restarts oder einen kontrollierten Abbruch abgebildet werden.
- **Interfaces und Reflection.** Dynamische Dispatch- und Reflection-Pfade sind nicht Teil eines affinen Speichermodells; sie brauchen die bestehende Objekt- und Klassenmaschinerie oder bleiben außerhalb der ersten Stufe.
- **`unsafe`, `cgo`, `runtime.SetFinalizer`.** Diese brechen jede statische Lebensdauer- und Finalisierungsgarantie. Eine erste Stufe müsste sie ausdrücklich ablehnen.

Eine realistische erste Stufe ist deshalb eine **Go-Teilmenge**:

- `go` wird zu `spawn`; jede Goroutine ist ein Actor mit eigener Epoche.
- `chan` wird zu einem store-residenten Channel; Nachrichten werden beim Senden promottet.
- `defer` wird zu einem LIFO-Stack von Finalizer-Tokens; `bracket` garantiert die Ausführung bei normalem Return, Fehler und Panic.
- Heap-Werte werden entweder in die Actor-Epoche allokiert oder, wenn sie die Goroutine verlassen, in den Image-Store promottet.
- Geteilter, mutabler Speicher über Goroutinen hinweg wird entweder verboten oder als expliziter Ownership-Transfer modelliert.
- `select` und blockierende Channel-Operationen werden auf den Scheduler abgebildet; kooperatives Yield ersetzt die Go-Präemption.
- `close` eines Channels ist ein linearer Finalizer-Token; die Channel-Semantik bleibt store-resident.

Damit wäre ein Go-Transpiler für eine **nebenläufige, kanalbasierte Teilmenge** denkbar, die keine beliebten Go-Idiome wie `sync.Mutex`, frei geteilte Heap-Zeiger, Reflection oder Finalizer benötigt. Ein vollständiger Go-Compiler bräuchte dagegen einen tracing-GC oder eine sehr viel stärkere Regionen- und Ownership-Inferenz; die affine Epoche allein reicht dafür nicht.

Offene Entscheidungen

1. **Lifetime:** Epoche pro Actor-Inkarnation (heute kompatibel) oder pro Turn (braucht Promotion bei `yield`, `halt` und `send`)?
2. **Nachrichten:** im Image lassen oder beim Senden in die Empfänger-Epoche kopieren?
3. **Repräsentation:** managed Array-Arena (weniger Allokationen, weiterhin GC) oder unmanaged Bump-Allocator mit Tensor-View (wirklich GC-frei, aber neue Safety- und AOT-Grenzen)?
4. **Durchsetzung:** dynamische Ownership-Prüfung (W9, D30) oder statische Contracts (D54)?
5. **Go-Teilmenge:** Welche Sprachfeatures sind in der ersten Stufe erlaubt, und welche werden mit einem benannten Fehler abgelehnt?

Die konservative Antwort ist: Der Image-Store bleibt die Autorität; eine per-Inkarnations-Epoche hält nur ephemere Zwischenwerte; Nachrichten und persistierte Werte werden promottet; Seiteneffekte laufen über `bracket` und einen linearen Finalizer-Token. Das ist kein bestehendes

Verhalten, sondern ein potenzieller nächster Speicher-Tier und eine mögliche Grundlage für einen eingeschränkten Go-Transpiler.